Fredrik Hammarberg

Optimizing Weak Reference Processing in the JVM Z Garbage Collector

A Performance Analysis and Optimization of the ZGC Reference Processing Pipeline

UPPSALA
UNIVERSITET

Abstract
PLACEHOLDER

Acknowledgments

I would like to express my sincere gratitude to my supervisor for their guidance and support throughout this thesis work. Their expertise in JVM internals and garbage collection algorithms has been invaluable in shaping this research.

Special thanks to the Department of Information Technology at Uppsala University for providing the resources and environment necessary to conduct this research.

Finally, I would like to thank my family and friends for their patience and encouragement throughout my studies.

Uppsala, Sweden, June 2026

Fredrik Hammarberg

Contents

Acknowledgments iii

1 Introduction 9
1.1 Motivation 9
1.2 Research Questions 9
1.3 Stretch Goals 9
1.4 Contributions 10
1.5 Thesis Structure 10

2 Background 11
2.1 Garbage Collection in the JVM 11
2.1.1 Basic Concepts 11
2.1.2 Stop-The-World vs Concurrent Collection 11
2.2 G1 Garbage Collector 11
2.2.1 Marking and Collection 11
2.3 Z Garbage Collector 11
2.3.1 Generational ZGC 11
2.4 Java Reference Types 11
2.4.1 WeakReference and ReferenceQueue 11
2.5 Reference Processing Pipeline 11
2.5.1 Discovery 11
2.5.2 Processing 11
2.5.3 Enqueue 11
2.6 Related Work 11
2.7 Summary 11

3 Method 12
3.1 Implementation Approach 12
3.1.1 ZGC Modifications 12
3.1.2 G1GC Modifications 12
3.2 Optimization Strategies 12
3.2.1 Null-Queue Skip Optimization 12
3.2.2 Data Structure Optimizations 12
3.2.3 Synchronization Optimizations 12
3.3 Benchmarking Methodology 12
3.3.1 Synthetic Benchmarks 12
3.3.2 Real-World Benchmarks 12

3.4 Experimental Setup 12
3.4.1 Hardware Configuration 12
3.4.2 Software Configuration 12
3.4.3 Measurement Methodology 12

4 Results 13
4.1 Synthetic Benchmark Results 13
4.1.1 Baseline Performance 13
4.1.2 Null-Queue Optimization Impact 13
4.1.3 Data Structure Impact 13
4.2 Caffeine Benchmark Results 13
4.2.1 Cache Operations Performance 13
4.2.2 GC Pause Time Analysis 13
4.3 Statistical Analysis 13
4.4 Summary of Findings 13

5 Discussion 14
5.1 Interpretation of Results 14
5.1.1 Performance Improvements 14
5.1.2 Trade-offs 14
5.2 Implications for JVM Development 14
5.2.1 Applicability to Other Collectors 14
5.2.2 Integration Challenges 14
5.3 Limitations 14
5.3.1 Benchmark Coverage 14
5.3.2 Correctness Considerations 14
5.3.3 Generational Complexity 14
5.4 Comparison with Related Work 14
5.5 Summary 14

6 Conclusion 15
6.1 Summary of Contributions 15
6.2 Answers to Research Questions 15
6.3 Future Work 15
6.3.1 Further Optimizations 15
6.3.2 Broader Evaluation 15
6.3.3 Integration with Mainline OpenJDK 15
6.4 Closing Remarks 15

List of Tables

List of Figures

1. Introduction

As the Java programming language moves further away from end-users and into the realm of large-scale enterprise applications, the performance of the Java Virtual Machine (JVM) becomes increasingly critical. Especially when it comes to performance on large heaps since many enterprise applications utilize large amounts of memory.

The Z Garbage Collector (ZGC) is a low-latency garbage collector designed to handle large heaps with minimal pause times. However, the processing of weak references in ZGC has been identified as a potential bottleneck, particularly in applications that heavily utilize weak references. This thesis aims to analyze the performance of weak reference processing in ZGC and explore potential optimizations to improve its efficiency.

1.1 Motivation

Because of the increasing usage of Java in server-side applications and other large-scale applications, the performance of the JVM is crucial. ZGC is designed to handle large heaps (100s of GBs) with low latency, making it suitable for such applications. However, the processing of weak references in ZGC can lead to performance issues, especially in applications that heavily utilize weak references. Understanding and optimizing this aspect of ZGC can lead to significant performance improvements for a wide range of applications.

1.2 Research Questions

The main research questions that this thesis aims to address are:

1. What design changes and optimizations can be made to the weak reference processing in ZGC to improve its performance?
2. How do these design changes and optimizations impact the overall performance and memory footprint of ZGC?

1.3 Stretch Goals

The following stretch goals are defined for this thesis:

1. Does the introduction of a weak field annotation in Java provide a more efficient way to handle weak references compared to the current implementation?
2. Can the optimizations developed for ZGC be applied to other garbage collectors in the JVM, such as the G1 garbage collector, and if so, what performance improvements can be observed?

1.4 Contributions

1.5 Thesis Structure

2. Background

2.1 Garbage Collection in the JVM

2.1.1 Basic Concepts

2.1.2 Stop-The-World vs Concurrent Collection

2.2 G1 Garbage Collector

2.2.1 Marking and Collection

2.3 Z Garbage Collector

2.3.1 Generational ZGC

2.4 Java Reference Types

2.4.1 WeakReference and ReferenceQueue

2.5 Reference Processing Pipeline

2.5.1 Discovery

2.5.2 Processing

2.5.3 Enqueue

2.6 Related Work

2.7 Summary

3. Method

3.1 Implementation Approach

3.1.1 ZGC Modifications

3.1.2 G1GC Modifications

3.2 Optimization Strategies

3.2.1 Null-Queue Skip Optimization

3.2.2 Data Structure Optimizations

3.2.3 Synchronization Optimizations

3.3 Benchmarking Methodology

3.3.1 Synthetic Benchmarks

3.3.2 Real-World Benchmarks

3.4 Experimental Setup

3.4.1 Hardware Configuration

3.4.2 Software Configuration

3.4.3 Measurement Methodology

4. Results

4.1 Synthetic Benchmark Results

4.1.1 Baseline Performance

4.1.2 Null-Queue Optimization Impact

4.1.3 Data Structure Impact

4.2 Caffeine Benchmark Results

4.2.1 Cache Operations Performance

4.2.2 GC Pause Time Analysis

4.3 Statistical Analysis

4.4 Summary of Findings

5. Discussion

5.1 Interpretation of Results

5.1.1 Performance Improvements

5.1.2 Trade-offs

5.2 Implications for JVM Development

5.2.1 Applicability to Other Collectors

5.2.2 Integration Challenges

5.3 Limitations

5.3.1 Benchmark Coverage

5.3.2 Correctness Considerations

5.3.3 Generational Complexity

5.4 Comparison with Related Work

5.5 Summary

6. Conclusion

6.1 Summary of Contributions

6.2 Answers to Research Questions

6.3 Future Work

6.3.1 Further Optimizations

6.3.2 Broader Evaluation

6.3.3 Integration with Mainline OpenJDK

6.4 Closing Remarks